Lesson 2: Derivatives, and numbers that remember

Slopes measured by nudging, why h cannot shrink forever, partial derivatives, and a Value class that records how it was computed.

Duration	About 1 h 40 min, plus homework.
Files	`01_python_warmup/derivative.py` and `01_python_warmup/value.py`, both empty at the start.
Before	Lesson 1 done. `import matplotlib` works in the `.venv`, otherwise run `uv pip install -r requirements.txt`. Have the learner's written answers from lesson 1, block 1, at hand.
Goals	The learner estimates a derivative numerically, explains why `h` cannot go to zero on a computer, reads a partial derivative as "nudge one input, freeze the others", explains why the derivative of `a * b + c` with respect to `a` is `b`, and builds a class whose `+` and `*` remember their inputs.

How to run this session

The rules from lesson 1 still hold.

- Ask for a prediction before every run.
- Give hints in order, only after a real attempt, and keep your hands off the keyboard.
- The learner reads errors aloud before anyone edits. Every message quoted here was checked on Python 3.14.
- Green boxes are for you.

Do not say "backpropagation" yet. The learner should find the idea before lesson 3 gives it a name.

Block	Time	Where
Warm-up	5 min	
1. The numerical derivative	40 min	`derivative.py`
2. Partial derivatives and `Value`	45 min	`value.py`
3. Wrap-up and commit	10 min	`README.md`, terminal

Warm-up (5 min)

> **ASK**
> Without looking at the code: why does `2 * u` fail while `u * 2` works, and what fixed it?

> **EXPECTED (FOR YOU)**
> `int.__mul__` returns `NotImplemented`, so Python tries `__rmul__` on the right object. If the answer is shaky, spend five more minutes on it. Block 2 relies on the same mechanism.

Block 1: the numerical derivative

All of block 1 goes in `derivative.py`. The function is $f(x) = 3x^2 - 4x + 5$.

Step 1: f in Python, and its derivative by hand

TASK
Write `f` as a Python function. The learner computes `f(3)` by hand before running it.

EXPECTED (FOR YOU)
`f(3)` gives `20`.

WATCH FOR
- `SyntaxError: invalid decimal literal` on `3x**2`. Python needs `3 * x**2`.
- `x^2` runs without an error and gives a wrong number. In Python `^` is bitwise XOR, not a power, so `10 ^ -15` is `-5`. This one bites again in step 4 when writing small values of `h`.

ASK
On paper: what is $f'(x)$, and what is $f'(3)$?

EXPECTED (FOR YOU)
$f'(x) = 6x - 4$, so $f'(3) = 14$. This is terminale material and should take a minute.

Step 2: measuring the slope without the formula

ASK
Using only `f`, with no formula for f', how could you measure the slope at $x = 3$?

IF STUCK, IN THIS ORDER
1. "A slope is rise over run. Pick two points very close together on the curve."
2. "Nudge `x` by a small `h`. How much did `f` move? Divide by `h`."

EXPECTED (FOR YOU)
`(f(3 + h) - f(3)) / h`. With `h = 0.001` it prints `14.00300000000243`.

ASK
Why 14.003 and not 14? Expand $f(3 + h)$ by hand.

EXPECTED (FOR YOU)
$f(3 + h) = 20 + 14h + 3h^2$, so the ratio is exactly $14 + 3h$ and the error is $3h$. Have the learner predict the result for `h = 0.01` and `h = 0.1` before running. The code prints `14.029999999999632` and `14.300000000000068`, which are 14.03 and 14.3 plus float noise.

Step 3: other points, and which way is downhill

ASK
Predict the slope at $x = -3$. Then find the x where the slope is zero.

EXPECTED (FOR YOU)
$f'(-3) = -22$, and the code gives `-21.997` with `h = 0.001`. The slope is zero at $x = \frac{2}{3}$, the bottom of the parabola, where the code gives `0.003`.

ASK

You stand at $x = 3$ and want f smaller. Do you step left or right? What told you?

EXPECTED (FOR YOU)

Left, because the slope is positive. The sign of the derivative points uphill, so you step the other way. This is gradient descent from chapter 2, in one dimension.

Step 4: shrinking h until it breaks

TASK

A loop over `h` from `0.1` down to `1e-16`, printing the estimate each time. Before running it, the learner predicts whether the estimate keeps getting closer to 14.

EXPECTED (FOR YOU)

h	Estimate of $f'(3)$
0.1	14.300000000000068
0.01	14.029999999999632
0.001	14.00300000000243
1e-06	14.000003002223593
1e-08	14.00000009255109
1e-10	14.000001158365194
1e-12	14.001244608152774
1e-14	14.210854715202004
1e-15	10.658141036401503
1e-16	0.0

ASK

The maths says a smaller `h` is always better. Why does the computer disagree?

IF STUCK, IN THIS ORDER

1. "Run `3 + 1e-16 == 3`." It prints `True`.
2. "A float keeps about 16 significant digits. Where does the last 1 of `3.0000000000000001` go?"

EXPECTED (FOR YOU)

Near 3, two neighboring floats are about `4.4e-16` apart, so `3 + 1e-16` rounds back to 3 and the numerator is 0. Around `h = 1e-14`, `f(3 + h)` and `f(3)` agree on almost every digit, and their difference keeps only one or two correct ones. The estimate is best near `h = 1e-8`, where the $3h$ error and the rounding error are about the same size.

Step 5: plot f and its tangent at x = 3

ASK

What is the equation of the tangent to f at $x = 3$?

EXPECTED (FOR YOU)

$y = f(3) + f'(3)(x - 3)$, so $y = 20 + 14(x - 3)$.

The learner writes a function `tangent(x)` from that equation. The plotting code is not what this lesson teaches, so hand it over:

```
# %% Plot f and its tangent at x = 3
import matplotlib.pyplot as plt

xs = [i / 10 for i in range(-20, 61)]  # x from -2 to 6, step 0.1
plt.plot(xs, [f(x) for x in xs], label="f")
plt.plot(xs, [tangent(x) for x in xs], label="tangent at x = 3")
plt.scatter([3], [f(3)], color="red", zorder=3)  # the point (3, 20)
plt.legend()
plt.grid(True)
plt.show()
```

EXPECTED (FOR YOU)

The line touches the parabola at (3, 20) and stays below it everywhere else. At $x = 4$ the gap is $37 - 34 = 3$, which is the $3h^2$ term from step 2 with $h = 1$.

Block 2: partial derivatives and the `Value` class

All of block 2 goes in `value.py`.

Step 1: three inputs

TASK

With plain floats, `a, b, c = 2.0, -3.0, 10.0` and `d = a * b + c`. The learner predicts `d` first.

EXPECTED (FOR YOU)

`4.0`

ASK

If you nudge `a` up a tiny bit and leave `b` and `c` alone, does `d` go up or down? By how much, compared to the nudge? Then the same questions for `b`, and for `c`.

IF STUCK, IN THIS ORDER

1. "It is the trick from `derivative.py`, but only one input moves."
2. "Compute `d` once, then again with `a + h` in place of `a`."

EXPECTED (FOR YOU)

With `h = 0.0001`, the three slopes print as:

- `-3.000000000010772` for `a`
- `2.0000000000042206` for `b`
- `0.9999999999976694` for `c`

Now give the idea its name. A derivative with respect to one input, with the others frozen, is a partial derivative. It is written $\partial d/\partial a$ and read "partial d over partial a".

ASK

Why is $\partial d/\partial a$ equal to b?

IF STUCK, IN THIS ORDER

1. "Freeze $b = -3$ and $c = 10$. Write d as a function of a alone."
2. "What is the slope of $d = -3a + 10$?"

EXPECTED (FOR YOU)

With b and c frozen, d is a straight line in a whose slope is b. In the same way, $\partial d/\partial b = a = 2$ and $\partial d/\partial c = 1$. To check the idea transfers, ask for $\partial d/\partial a$ when $b = 5$, without code. The answer is 5.

Step 2: why Python should remember

ASK

How many times did you compute `d` to get the three slopes? How many would a network with 13,002 weights need for one learning step?

EXPECTED (FOR YOU)

Four, one baseline and one per input. The network would need 13,003 full runs for a single nudge of its weights. If Python remembered how `d` was built, the derivatives could come from that record instead. Do not explain how. That is lesson 3.

Step 3: the `Value` class

TASK

Write a class `Value` that stores a number, the `Value` objects it was made from, and the operation that made it. Inputs have no parents and an empty operation. It needs `__init__`, `__repr__`, `__add__` and `__mul__`. The attribute names in the checks below match micrograd, which makes the lesson 3 video easier to follow.

```
# %% Checks
a = Value(2.0)
b = Value(-3.0)
c = Value(10.0)
e = a * b
d = e + c
print(d)        # Value(data=4.0)
print(d._op)    # +
print(d._prev)  # the two parents, Value(data=-6.0) and Value(data=10.0)
print(a._prev)  # nothing, because a is an input
```

IF STUCK, IN THIS ORDER

1. "Your `Vector2D.__add__` returned a new vector. What should `Value.__add__` return, and what extra information should it pass along?"
2. "`Value(2.0)` has no parents. How do you give a parameter a default value in Python?"

WATCH FOR

- `d._prev` shows plain numbers such as `-6.0` and `10.0`. The method stored `other.data` instead of `other`. Ask: "Starting from `d`, can you get back to `a`?" With numbers, the trail stops after one step.
- `print(d)` shows `<__main__.Value object at ...>`. `__repr__` is missing, and lesson 1 covered it.

ASK

Back to the wrap-up question of lesson 1. When Python runs `a * b + c` on `Value` objects, what gets built?

EXPECTED (FOR YOU)

`__mul__` builds `e` with parents `a` and `b`, then `__add__` builds `d` with parents `e` and `c`. Writing an ordinary formula builds a graph of the computation, with no extra work from the learner.

Step 4, stretch: print the graph

TASK

Add a `label` to `Value`, as micrograd does, so that `Value(2.0, label="a")` works and `e.label = "e"` can be set after `e = a * b`. Then write a function that prints how `d` was built, one line per `Value`, indented by depth. The order of siblings may differ:

```
d = 4.0 (+)
  e = -6.0 (*)
    a = 2.0
    b = -3.0
  c = 10.0
```

IF STUCK, IN THIS ORDER

1. "What should the printer do with a `Value` that has no parents?"
2. "For `d`, it prints one line. What job is left for each parent? Is it the same job?"
3. "A function may call itself. Pass `depth + 1` down."

Block 3: wrap-up and commit

Bring out the answers from lesson 1, block 1, and ask the same two questions again.

ASK

1. What does learning change inside the network?
2. Why do we need derivatives?

EXPECTED (FOR YOU)

The answers should now draw on this lesson. In `d = a * b + c`, think of `a`, `b` and `c` as weights and `d` as the cost. $\partial d/\partial a = -3$ says that raising `a` lowers `d` three times as fast, so learning would raise `a`. A network does the same for each of its 13,002 weights. Compare with the first answers out loud, and let the learner say what changed.

```
git add 01_python_warmup/derivative.py 01_python_warmup/value.py
git commit -m "derivative and value: numerical slopes, float limits, partial derivatives"
git push
```

Then a few sentences in the `README.md` journal, committed too.

Homework

All in `value.py`, in this order. Each task makes the next one easier.

1. Make `Value(2.0) + 3` and `Value(2.0) * 3` work by wrapping plain numbers in a `Value`. Hint if needed: `isinstance`.
2. Make `3 + Value(2.0)` and `3 * Value(2.0)` work. This is the `2 * u` puzzle again, and the learner should recognize it without help.
3. `__sub__`, so that `Value(5.0) - Value(3.0)` gives `Value(data=2.0)`. It can reuse `+` and `*` with `-1`, which is why tasks 1 and 2 come first.

4. `__pow__` with a plain number as exponent, so that `Value(3.0) ** 2` gives `Value(data=9.0)`. Ask whether the exponent belongs in the parents. It does not, because it is a plain number and not a `Value`.
5. `tanh()`, using $\tanh(x) = (e^{2x} - 1)/(e^{2x} + 1)$ and `math.exp`. `Value(0.0).tanh()` gives `0.0`, and `Value(0.8813735870195432).tanh()` gives `0.7071067811865476`. Before comparing a few values with `math.tanh`, the learner predicts `tanh` at 3 and at −3. The answers are about 0.995 and −0.995.

Next: lesson 3 starts Karpathy's micrograd video. Its first half hour repeats this lesson almost step by step.